

5.1 - Versionamento semântico e ciclo de vida de APIs

Como comunicar mudanças sem quebrar quem depende de você

Uma 'melhoria' que tirou três transportadoras do ar

- **API pública de tracking integrada com três transportadoras parceiras**
 - Mudança classificada internamente como melhoria de nomenclatura

LUNALOG

Numa quarta-feira de manhã, a LunaLog publicou uma atualização que renomeou o campo status para delivery_status na API de tracking. Sem aviso, sem versão nova, sem período de transição. Em poucas horas, três transportadoras pararam de funcionar ao mesmo tempo, o campo que elas liam simplesmente havia sumido. Roberto passou dois dias coordenando suporte emergencial. Ana escreveu uma carta de desculpas formal. O custo invisível foi a confiança das parceiras, que nunca mais aceitaram integração sem cláusula contratual de versionamento.

Versionamento semântico, a gramática das mudanças

Três números que comunicam o tipo de impacto

- **SemVer usa o formato MAJOR.MINOR.PATCH para comunicar impacto de cada mudança**
 - MAJOR muda quando há quebra de compatibilidade com versões anteriores
 - MINOR muda quando há nova funcionalidade compatível com versões antigas
 - PATCH muda quando há correção que não altera comportamento esperado
- **O contrato é com quem consome, não com quem desenvolve internamente**
 - Refatoração interna sem mudança de interface é PATCH, não MINOR
- **Bumps de MAJOR são raros e precisam ser comunicados com antecedência**
 - Cada MAJOR representa custo de migração para todos os consumidores
- **Versões pré-release usam sufixos como alpha, beta ou rc para sinalizar instabilidade**

Onde colocar a versão de uma API HTTP

Três estratégias com trade-offs distintos

- **URL path como /v1/orders, a estratégia mais visível e cacheável**
 - Versão explícita em logs, métricas e rotas de gateway
 - *Mistura conceitualmente identidade de recurso com identidade de contrato*
- **Header HTTP customizado como Accept-Version, separa contrato de identidade**
 - Mais limpo conceitualmente, porém menos óbvio para clientes humanos testando
- **Query string como ?version=1, opção mais frágil e menos recomendada**
 - Pode ser ignorada em cache, esquecida em integração ou removida em refatoração
- **Escolha precisa ser única na organização, consistência supera perfeição**

Quebra silenciosa contra mudança comunicada

Quebra silenciosa (o caminho fácil)

- Campo renomeado sem aviso prévio
- Resposta com estrutura alterada de um deploy para outro
- Cliente descobre a mudança em produção, falhando
- Suporte emergencial vira norma, não exceção
- Confiança das integrações se deteriora ao longo do tempo

Mudança comunicada (o caminho profissional)

- Nova versão coexiste com a antiga por tempo definido
- Anúncio prévio com sunset date no calendário
- Header de depreciação avisa em cada resposta da versão antiga
- Cliente migra dentro da janela acordada, sem urgência
- Confiança se constrói exatamente pela previsibilidade

Backward compatibility é compromisso, não cortesia

- **Toda mudança de API afeta sistemas que você não controla mais**
- **Backward compatibility significa que a versão antiga continua respondendo corretamente**

Quando um time decide quebrar compatibilidade, está delegando custo de migração para terceiros. Essa transferência de custo precisa ser uma decisão arquitetural consciente, com janela, comunicação e suporte, jamais um efeito colateral de refatoração.

- O custo de manter duas versões em paralelo é menor que o custo de perder integrações

Ciclo de vida, depreciação e sunset

A morte planejada de uma versão

- **Toda versão de API nasce com data de morte prevista**
 - Deprecation date marca quando a versão entra em manutenção mínima
 - Sunset date marca quando a versão deixa de responder definitivamente
- **Headers Deprecation e Sunset comunicam o ciclo em cada resposta da API**
 - RFC 8594 padroniza o header Sunset com data ISO 8601
- **Comunicação ativa vale mais que header, e-mail e changelog precisam reforçar**
 - Quanto mais críticos os consumidores, mais antecipada precisa ser a comunicação
- **Duas versões ativas têm custo, três indicam falha de processo**



Versionamento semântico define como comunicar mudanças ao mundo. Mas como garantir que as mudanças não quebrem silenciosamente quem depende delas, especialmente em microsserviços onde times diferentes controlam cada lado da integração? Existe um modelo que inverte a lógica tradicional, em vez do provedor anunciar o que oferece, o consumidor declara o que espera.

Consumer-driven contracts, quando quem consome dita o contrato